3D Portfolio Project Documentation

Detailed implementation guide for Dinesh Reddy Ande's React, Vite, Three.js, GSAP, and Tailwind CSS portfolio.

Area	Details
Project	three-d-portfolio-2025
Framework	React 19 with Vite
UI and styling	Tailwind CSS 4 plus custom CSS layers
3D rendering	Three.js, React Three Fiber, and Drei
Animation	GSAP and ScrollTrigger
Profile focus	Backend systems, cloud engineering, AI/RAG, and full-stack projects

> This PDF explains how the portfolio works from start to finish. It covers the runtime flow, file structure, major sections, data model, animation logic, 3D rendering, styling system, and practical commands for running and building the app.

1. Executive Summary

This project is a single-page interactive 3D portfolio application. It uses Vite as the development and build tool, React as the UI framework, Tailwind CSS for layout and visual styling, GSAP for entrance and scroll-triggered animation, and React Three Fiber/Drei for 3D rendering. The site has been personalized around Dinesh Reddy Ande's software engineering profile, including experience at Infosys and OneAdvanced, graduate study at the University of Florida, backend systems, cloud platforms, and AI/RAG work.

The application is component-based. Each major page area is isolated in a section component, while reusable pieces such as the navbar, buttons, glowing cards, counters, and 3D model cards live under src/components. Most profile content is centralized in src/constants/index.js, which makes the project easier to maintain because text, metrics, experience items, tech labels, and social links can be edited without rewriting section logic.

- Browser tab branding with Dinesh's name and a custom DR favicon.
- Responsive navigation with scroll-aware styling and a direct email action.
- Hero section with animated rotating keywords and an interactive 3D room scene.
- Resume-based counters for experience, transaction scale, API improvement, and ML precision.
- Text-first project showcase with structured project cards instead of screenshots.
- Animated technology marquee, feature cards, work timeline, 3D tech cards, and footer links.

2. Technology Stack and Responsibility

Technology	Where it appears	Responsibility
Vite	package.json, vite.config.js	Development server, hot reload, production build, and preview workflow.
React 19	src/main.jsx, src/App.jsx, section/ components	Composes the portfolio from reusable UI components.
Tailwind CSS 4	src/index.css	Controls layout, spacing, typography, color tokens, responsive behavior, and component styling.
GSAP + ScrollTrigger	Hero, counters, projects, experience, skills	Runs entrance animations, scroll-triggered reveals, counters, and timeline effects.
React Three Fiber + Drei	HeroExperience, TechIconCardExperience	Renders 3D scenes and GLB models inside React components.
Static assets	public/images, public/models	Stores icons, background imagery, favicon, and optimized GLB models.

3. Project Structure

The project follows a standard Vite React layout. Static assets live in public, React source code lives in src, and the application is split into constants, components, sections, and model-specific components.

```
3d-portfolio/
+-- index.html
+-- package.json
+-- vite.config.js
+-- public/
|   +-- images/
|   |   +-- fav.svg
|   +-- models/
|       +-- optimized-room.glb
|       +-- react_logo-transformed.glb
|       +-- python-transformed.glb
+-- src/
    +-- main.jsx
    +-- App.jsx
    +-- index.css
    +-- constants/index.js
    +-- components/
    +-- sections/
```

4. Browser Entry and React Bootstrapping

The browser first loads index.html. That file defines the favicon, tab title, root DOM node, and script entry point. The visible interface is not written directly in index.html; it is mounted by React.

```
<link rel="icon" type="image/svg+xml" href="/images/fav.svg" />
<title>Dinesh Reddy Ande - Portfolio</title>

<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

Code: index.html

React entry point

src/main.jsx imports the global stylesheet and renders the App component into the root element. StrictMode is enabled so React can surface potential development issues.

```
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Code: src/main.jsx

5. Application Composition

App.jsx is the top-level page layout. It imports every visible section and renders them in the exact order users see on the page. This makes the page easy to reason about: if a section needs to be removed, moved, or replaced, App.jsx is the first place to look.

```
const App = () => (
  <>
    <Navbar />
    <Hero />
    <ShowcaseSection />
    <LogoShowcase />
    <FeatureCards />
    <Experience />
    <TechStack />
    <Footer />
  </>
);
```

The contact section and the education/highlight section were removed by deleting their imports and JSX tags from this file. The remaining flow is concise and focused on the strongest parts of the portfolio.

Code: src/App.jsx

6. Central Data Model

The constants file is the main content source for the portfolio. It stores navigation labels, animated hero words, counter values, skill lists, experience details, and footer links. Keeping content here prevents the UI components from becoming hardcoded and repetitive.

Navigation data

```
const navLinks = [
  { name: "Projects", link: "#work" },
  { name: "Experience", link: "#experience" },
  { name: "Skills", link: "#skills" },
];
```

Resume-based counters

```
const counterItems = [
  { value: 3, suffix: "+", label: "Years of Software Engineering Experience" },
  { value: 1, suffix: "M+", label: "Peak Daily Transactions Supported" },
  { value: 70, suffix: "%", label: "API Throughput Improvement" },
  { value: 98, suffix: "%", label: "ML Crack Detection Precision" },
];
```

Experience records

```
{
  review:
    "Modernized an account opening platform with compliant, event-driven services that improved
processing efficiency by 40%.",
  company: "Infosys",
  logoText: "INFY",
  role: "Specialist Programmer L2",
  date: "July 2024 - August 2025",
  responsibilities: [
    "Built microservice architecture with Java, Spring Boot, Kafka, and Redis...",
    "Improved API throughput by 70%...",
  ],
}
```

Code: src/constants/index.js

---

7. Navbar Implementation

The navbar displays the site owner name, section navigation, and an email button. It also listens for scroll events and changes its styling after the page has been scrolled more than ten pixels.

```
const [scrolled, setScrolled] = useState(false);

useEffect(() => {
  const handleScroll = () => {
    const isScrolled = window.scrollY > 10;
    setScrolled(isScrolled);
  };

  window.addEventListener("scroll", handleScroll);
  return () => window.removeEventListener("scroll", handleScroll);
}, []);
```

The component then conditionally assigns either scrolled or not-scrolled to the header. CSS uses those classes to switch the navbar between a transparent state and a black background.

```
<header className={`navbar ${scrolled ? "scrolled" : "not-scrolled"}`}>
  ...
</header>
```

The contact button is a direct email link. This is appropriate because the full contact form section was removed from the current site.

```
<a href="mailto:dineshreddy.ande@ufl.edu" className="contact-btn group">
  <div className="inner">
    <span>Email me</span>
  </div>
</a>
```

Code: src/components/NavBar.jsx

---

8. Hero Section

The hero section introduces Dinesh and creates the strongest first impression. It contains a large animated headline, rotating keywords, an intro paragraph, a call-to-action button, the 3D room scene, and the animated counters below.

```
<div className="hero-text">
  <h1>
    Shaping
    <span className="slide">
      <span className="wrapper">
        {words.map((word, index) => (
          <span key={index} className="flex items-center md:gap-3 gap-1 pb-2">
            <img src={word.imgPath} alt="person" />
            <span>{word.text}</span>
          </span>
        ))}
      </span>
    </span>
  </h1>
  <h1>into Real Projects</h1>
  <h1>that Deliver Results</h1>
</div>
```

The rotating words are read from constants. CSS moves the wrapper vertically with a keyframe animation, so words like Backend, AI Systems, Cloud APIs, and Products appear inside the headline.

```
gsap.fromTo(
  ".hero-text h1",
  { y: 50, opacity: 0 },
  { y: 0, opacity: 1, stagger: 0.2, duration: 1, ease: "power2.inOut" }
);
```

The intro sentence is constrained with a dedicated hero-intro class. This keeps the paragraph aligned under the headline instead of stretching too wide.

```
.hero-intro {
  @apply max-w-[28ch] md:max-w-[38ch] xl:max-w-[42ch] leading-relaxed;
}
```

Code: src/sections/Hero.jsx

Styles: src/index.css

---

9. Hero 3D Scene

The hero 3D scene is built with React Three Fiber. Canvas creates a WebGL rendering surface. OrbitControls allows controlled rotation, while lights and the room model create the visual scene.

```
<Canvas camera={{ position: [0, 0, 15], fov: 45 }}>
  <ambientLight intensity={0.2} color="#1a1a40" />
  <OrbitControls
    enablePan={false}
    enableZoom={!isTablet}
    maxDistance={20}
    minDistance={5}
    minPolarAngle={Math.PI / 5}
    maxPolarAngle={Math.PI / 2}
  />

  <Suspense fallback={null}>
    <HeroLights />
    <group scale={isMobile ? 0.7 : 1} position={[0, -3.5, 0]}>
      <Room />
    </group>
  </Suspense>
</Canvas>
```

The component uses media queries to adjust the scene for mobile and tablet screens. The falling particle effect was removed by removing the Particles import and render call, while keeping the room and lighting intact.

Code: src/components/models/hero_models/HeroExperience.jsx

10. Animated Counter Metrics

AnimatedCounter renders four metric cards and animates the values when the counter section enters the viewport. Each counter has a DOM ref so GSAP can update the displayed number.

```
gsap.to(numberElement, {
  innerText: item.value,
  duration: 2.5,
  ease: "power2.out",
  snap: { innerText: 1 },
  scrollTrigger: {
    trigger: "#counter",
    start: "top center",
  },
  onComplete: () => {
    numberElement.textContent = `${item.value}${item.suffix}`;
  },
});
```

This implementation keeps the animation numeric during the count, then applies the suffix at completion. That is why values such as 1M+, 70%, and 98% display cleanly after the animation finishes.

Code: src/components/AnimatedCounter.jsx

11. Project Showcase

The project section was redesigned to avoid using placeholder screenshots. It now uses text-first case-study cards with project numbers, category labels, descriptions, metric callouts, and tech stack tags.

```
const projects = [
  {
    number: "01",
    title: "Agentic Multimodal RAG Assistant",
    kicker: "Grounded AI assistant",
    description:
      "A local-first chatbot with authenticated document upload, voice input, source-cited
chat, session memory, web-search fallback, ChromaDB retrieval, and cross-encoder reranking.",
    metric: "Source-cited answers",
    stack: ["FastAPI", "Streamlit", "LangChain", "ChromaDB"],
  },
];
```

The first project is featured on the left. Projects 2 and 3 are stacked on the right and use the compact card variant to reduce text size and spacing.

```
<ProjectCard project={projects[0]} featured />
<ProjectCard project={projects[1]} />
<ProjectCard project={projects[2]} />
```

```
const ProjectCard = ({ project, featured = false }) => (
  <article className={`project-card ${featured ? "featured" : "compact"}`}>
    <div className="project-card-top">
      <span>{project.number}</span>
      <p>{project.kicker}</p>
    </div>
    <div className="project-card-body">
      <h2>{project.title}</h2>
      <p>{project.description}</p>
    </div>
    <div className="project-card-footer">
      <strong>{project.metric}</strong>
      <div>
        {project.stack.map((tech) => (
          <span key={tech}>{tech}</span>
        ))}
      </div>
    </div>
  </article>
);
```

GSAP also animates the project cards into view. Each card starts slightly lower and transparent, then slides up and fades in when scrolled near the viewport.

Code: src/sections/ShowcaseSection.jsx

Styles: src/index.css

---

12. Skills Marquee

The marquee displays technology names in a continuous horizontal scroll. The list is rendered twice so the animation can loop smoothly without leaving an empty gap.

```
{logoIconsList.map((icon, index) => (
  <LogoIcon key={index} icon={icon} />
))}

{logoIconsList.map((icon, index) => (
  <LogoIcon key={index} icon={icon} />
))}
```

```
.marquee-box {
  display: flex;
  align-items: center;
  width: 200%;
  animation: marquee 60s linear infinite;
}

@keyframes marquee {
  0% { left: 0; }
  100% { left: -100%; }
}
```

Code: src/sections/LogoShowcase.jsx

13. Feature Cards

FeatureCards renders three summary cards from the abilities array. These cards communicate the main value areas of the portfolio: backend systems at scale, applied AI/RAG, and cloud delivery discipline.

```
{abilities.map(({ imgPath, title, desc }) => (
  <div key={title} className="card-border rounded-xl p-8 flex flex-col gap-4">
    <div className="size-14 flex items-center justify-center rounded-full">
      <img src={imgPath} alt={title} />
    </div>
    <h3 className="text-white text-2xl font-semibold mt-2">{title}</h3>
    <p className="text-white-50 text-lg">{desc}</p>
  </div>
))}
```

Code: src/sections/FeatureCards.jsx

14. Work Experience Timeline

The experience section maps over expCards and creates a timeline entry for each role. The left side uses a glowing card with a company identity block. The right side shows the company, role, date, and bullet responsibilities.

```
{expCards.map((card) => (
  <div key={card.title} className="exp-card-wrapper">
    <div className="xl:w-2/6">
      <GlowCard card={card}>
        <div className="company-panel-brand">
          <div className="company-panel-mark">{card.logoText}</div>
          <div>
            <p>{card.company}</p>
            <span>{card.role}</span>
          </div>
        </div>
      </GlowCard>
    </div>
  </div>
))}
```

```
<p className="text-[#839CB5] font-semibold uppercase tracking-wide">
  {card.company}
</p>
<h1 className="font-semibold text-3xl">{card.role}</h1>
<p className="my-5 text-white-50">{card.date}</p>

<ul className="list-disc ms-5 mt-5 flex flex-col gap-5 text-white-50">
  {card.responsibilities.map((responsibility, index) => (
    <li key={index} className="text-lg">
      {responsibility}
    </li>
  ))}
</ul>
```

ScrollTrigger is used heavily here. Timeline cards animate from the left, text blocks fade in, and the vertical timeline line scales as the user scrolls through the section.

```
gsap.utils.toArray(".timeline-card").forEach((card) => {
  gsap.from(card, {
    xPercent: -100,
    opacity: 0,
    transformOrigin: "left left",
    duration: 1,
    ease: "power2.inOut",
    scrollTrigger: {
      trigger: card,
      start: "top 80%",
    },
  });
});
```

Code: src/sections/Experience.jsx

Data: src/constants/index.js

15. GlowCard Interaction

GlowCard creates an interactive border glow based on mouse position. It calculates the pointer angle relative to the card center and writes that angle into a CSS variable. CSS then uses that variable in a conic gradient mask.

```
const rect = card.getBoundingClientRect();
const mouseX = e.clientX - rect.left - rect.width / 2;
const mouseY = e.clientY - rect.top - rect.height / 2;

let angle = Math.atan2(mouseY, mouseX) * (180 / Math.PI);
angle = (angle + 360) % 360;

card.style.setProperty("--start", angle + 60);
```

```
.card::before {
  background: var(--gradient);
  mask: linear-gradient(#0000, #0000),
    conic-gradient(
      from calc((var(--start) - 15) * 1deg),
      #ffffff1f 0deg,
      white,
      #ffffff00 100deg
    );
  opacity: 0;
}

.card:hover::before {
  opacity: 1;
}
```

Code: src/components/GlowCard.jsx

16. 3D Tech Stack Section

The skills section renders five 3D tech cards from the techStackIcons array. Each card contains a GLB model, a hover background animation, and a text label describing the skill area.

```
{techStackIcons.map((techStackIcon) => (
  <div
    key={techStackIcon.name}
    className="card-border tech-card overflow-hidden group xl:rounded-full rounded-lg"
  >
    <div className="tech-card-animated-bg" />
    <div className="tech-card-content">
      <div className="tech-icon-wrapper">
        <TechIconCardExperience model={techStackIcon} />
      </div>
      <div className="padding-x w-full">
        <p>{techStackIcon.name}</p>
      </div>
    </div>
  </div>
))}
```

The actual model rendering is delegated to TechIconCardExperience. This keeps TechStack focused on layout, while the 3D component handles lights, scene loading, floating animation, and OrbitControls.

```
const scene = useGLTF(model.modelPath);

<Float speed={5.5} rotationIntensity={0.5} floatIntensity={0.9}>
  <group scale={model.scale} rotation={model.rotation}>
    <primitive object={scene.scene} />
  </group>
</Float>
```

Code: src/sections/TechStack.jsx

3D model renderer: src/components/models/tech_logos/TechIconCardExperience.jsx

---

17. Footer and External Links

The footer renders social links from the socialImgs array. It supports normal external links and mailto links. External links open in a new tab, while email opens the user's mail client.

```
{socialImgs.map((socialImg, index) => (
  <a
    key={index}
    className="icon"
    href={socialImg.link}
    target={socialImg.link.startsWith("mailto:") ? undefined : "_blank"}
    rel="noreferrer"
    aria-label={socialImg.name}
  >
    <img src={socialImg.imgPath} alt="" />
  </a>
))}
```

Code: src/sections/Footer.jsx

---

18. Styling System

The styling system is centralized in src/index.css. It imports the font, loads Tailwind, defines custom colors, and groups reusable utilities and component styles. This file controls almost all visual behavior in the app.

```
@import url("https://fonts.googleapis.com/css2?
family=Mona+Sans:ital,wght@0,200..900;1,200..900&display=swap");
@import "tailwindcss";

@theme {
  --font-sans: "Mona Sans", sans-serif;
  --color-white-50: #d9ecff;
  --color-black-50: #1c1c21;
  --color-black-100: #0e0e10;
  --color-black-200: #282732;
}
```

Tailwind layers are used to define utilities and component styles. This makes repeated classes like flex-center, section-padding, grid helpers, navbar styles, project card styles, and footer styles reusable.

```
@layer utilities {
  .flex-center {
    @apply flex justify-center items-center;
  }
}

@layer components {
  .section-padding {
    @apply px-5 md:px-10 md:mt-40 mt-20;
  }
}
```

Code: src/index.css

19. Responsiveness

Responsiveness is handled through Tailwind breakpoint prefixes and a small amount of runtime screen detection for 3D rendering. Tailwind classes such as md:, xl:, and 2xl: control grids, spacing, font sizes, card heights, and section layout.

For the 3D hero scene, react-responsive is used to detect mobile and tablet screens. That affects room scale and whether zoom is enabled in OrbitControls.

```
const isMobile = useMediaQuery({ query: "(max-width: 768px)" });
const isTablet = useMediaQuery({ query: "(max-width: 1024px)" });
```

20. Running, Building, and Verifying the App

The app is managed with npm scripts defined in package.json.

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview"
}
```

Install dependencies

```
npm install
```

Start local development server

```
npm run dev
```

Create a production build

```
npm run build
```

Preview the production build

```
npm run preview
```

Run linting

```
npm run lint
```

21. Important Customizations Already Implemented

- Changed the browser tab from the template name to Dinesh Reddy Ande - Portfolio.
- Added a custom DR favicon at public/images/fav.svg.
- Personalized the hero, counters, skills, projects, experience, and footer links with resume-based details.
- Removed the contact form and replaced the navbar CTA with a direct email link.
- Removed the education/highlight section to keep the page focused.
- Rebuilt the project area into text-first case-study cards without screenshots.
- Changed work experience panels and timeline marks to show Infosys and OneAdvanced instead of template companies.
- Removed the falling particle effect from the hero scene.

22. Recommended Future Improvements

1. Add real GitHub and live demo links to each project card once the repositories or deployments are ready to share.
2. Replace inherited placeholder social icon images with precise GitHub, email, and LinkedIn SVG icons.
3. Add a resume download button in the hero or navbar.
4. Code-split the 3D sections with dynamic imports to reduce the initial JavaScript bundle size.
5. Audit all image alt text and aria labels before deploying publicly.
6. Optimize GLB model sizes periodically to protect page speed.
7. Consider adding lightweight project detail pages if you want each project to tell a deeper story.

23. Final Summary

The portfolio is a modern, interactive React application with strong visual presentation and maintainable code organization. App.jsx controls section order, constants/index.js controls profile content, section components render major page areas, reusable components provide shared UI behavior, and specialized 3D components render WebGL scenes and GLB models.

The implementation now reflects Dinesh Reddy Ande's actual profile rather than the original template. It emphasizes scalable backend systems, AI/RAG applications, cloud delivery, financial software experience, selected projects, and professional engineering impact.

Generated from local project path: /Users/dinesh/Project/3d-portfolio